



MFE 5210

Algorithmic Trading Basics (I)





Outline

- Introduction to MFE5210
 - Algo Trading Basics I
 - Git
-



Introduction to MFE5210

- Focus more on coding instead of math
- C++, Python
- No compulsory text book
- English + Chinese if necessary
- DON'T ASK ME WHAT IS WINNING STRATEGY
- Project will be similar with the MFE5250, but need to focus more on backtesting system, and project management



Algo Trading

- Algorithmic Trading 算法交易
 - Algorithms + Trading: Apply algorithms for trading
 - Algorithmic trading is a process for executing orders utilizing automated and pre-programmed trading instructions to account for variables such as price, timing and volume. An algorithm is a set of directions for solving a problem. Computer algorithms send small portions of the full order to the market over time.
-



Algo Trading

- Algorithmic Trading 算法交易
 - Algorithms + Trading: Apply algorithms for trading
 - Algorithmic trading makes use of complex formulas, combined with mathematical models and human oversight, to make decisions to buy or sell financial securities on an exchange. Algorithmic traders often make use of high-frequency trading technology, which can enable a firm to make tens of thousands of trades per second. Algorithmic trading can be used in a wide variety of situations including order execution, arbitrage, and trend trading strategies.
-



Algo Trading

✈ Trading

- ✈ Stock
- ✈ Bond
- ✈ Options and futures, other derivatives
- ✈ Others, e.g. mutual funds, btc





Algo Trading

- Algo --- Time Horizon
 - Long only
 - Low freq
 - High freq
 - Ultra high freq, low latency
 - Event driven (random holding period)
-



Algo Trading

→ Algo --- Strategies

- fundamental
- Quant
- Others, quantamental





Algo Trading

- Algorithmic trading is the use of process- and rules-based algorithms to employ strategies for executing trades.
 - It has grown significantly in popularity since the early 1980s and is used by institutional investors and large trading firms for a variety of purposes.
-

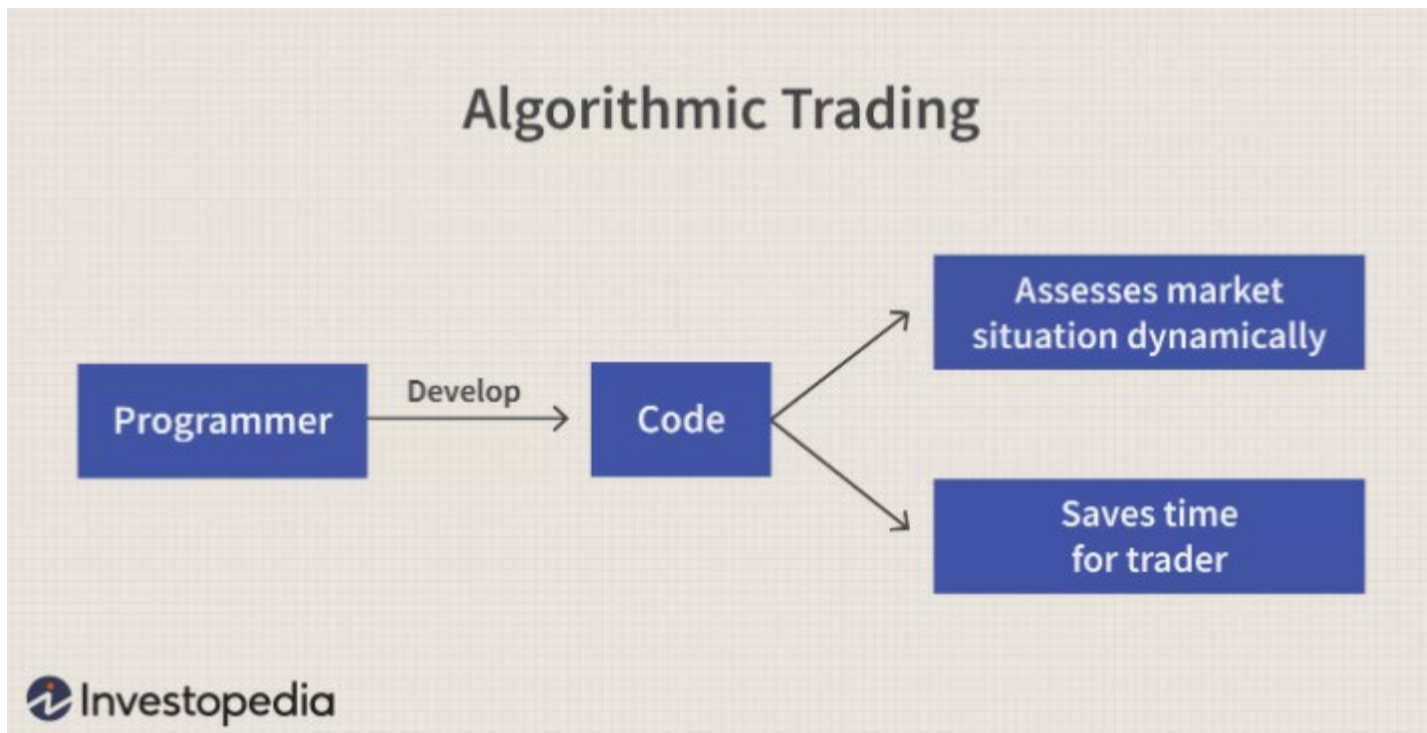


Algo Trading

- While it provides advantages, such as faster execution time and reduced costs, algorithmic trading can also exacerbate the market's negative tendencies by causing flash crashes and immediate loss of liquidity.
- ➔ Automated trading? High frequency trading?



Algo Trading





Algo Trading Benefits

- Trades are executed at the best possible prices.
 - Trade order placement is instant and accurate (there is a high chance of execution at the desired levels).
 - Trades are timed correctly and instantly to avoid significant price changes.
 - Reduced transaction costs.
-



Algo Trading Benefits

- Simultaneous automated checks on multiple market conditions.
 - Reduced risk of manual errors when placing trades.
 - Algo-trading can be backtested using available historical and real-time data to see if it is a viable trading strategy.
 - Reduced the possibility of mistakes by human traders based on emotional and psychological factors.
-

Concern on Algo Trading

- The speed of order execution, an advantage in ordinary circumstances, can become a problem when several orders are executed simultaneously without human intervention. The flash crash of 2010 has been blamed on algorithmic trading.





Concern on Algo Trading

- Another disadvantage of algorithmic trades is that liquidity, which is created through rapid buy and sell orders, can disappear in a moment, eliminating the chance for traders to profit off price changes. It can also lead to instant loss of liquidity. Research has uncovered that algorithmic trading was a major factor in causing a loss of liquidity in currency markets after the Swiss franc discontinued its Euro peg in 2015



System Design

- System (Project/Application) Design Environments (Stages), or Deployment Process
 - Requirement Analysis, ROLE: Product Manager
 - Development, ROLE: developer
 - Testing, ROLE: Test Engineer
 - Staging, *optional*, ROLE: Developer, Test Engineer, DevOps
 - Live/Production, ROLE: Developer, Test Engineer, DevOps
 - If something wrong in Production, go to re-development, re-testing, re-staging, and re-production. (Re-deployment)
 - Different process should be run in a **SEPARATE** environment (conda/virtualenv/docker), even machine
-



System Design

→ Front End Sub System

- For GUI (Graphical User Interface): Html/css/javascript rendering
- e.g. PnL Display system
- Even though its automated trading system, human monitoring is still necessary

→ Back End Sub System

- Database. Market Access, Connectivity, Order Routing
 - e.g. market data, order management system
-

Traditional Algo Trading System

- Any trading system, conceptually, is nothing more than a computational block that interacts with the exchange on two different streams.
 - Receives market data
 - Sends order requests and receives replies from the exchange.





Traditional Algo Trading System

→ Market data :

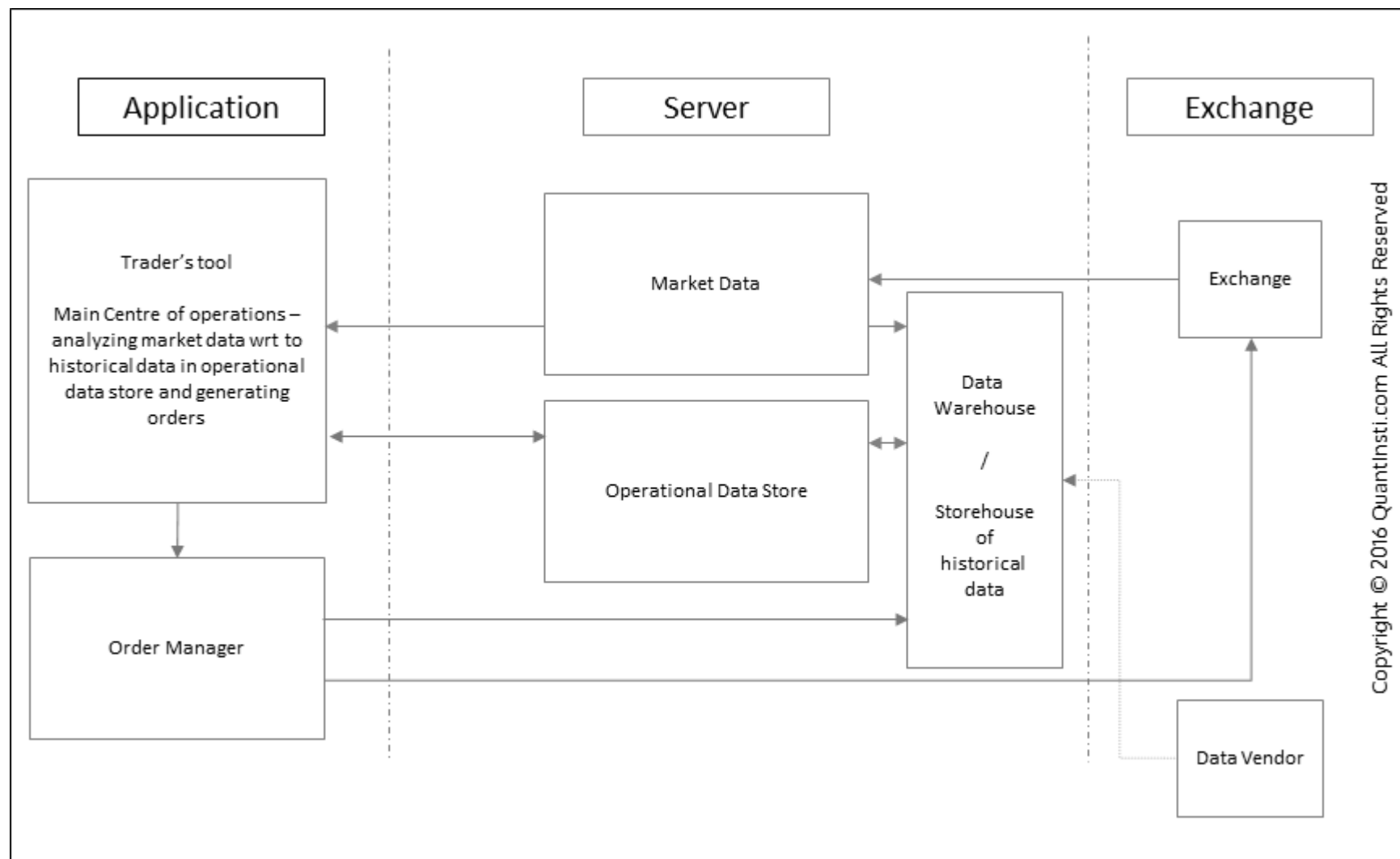
- informs the automated trading system of the latest order book.
 - It might contain some additional information like the volume traded so far, the last traded price and quantity for a scrip.
 - To cater to that, a conventional system would have a historical database to store the market data and tools to use that database.
 - The analysis would also involve a study of the past trades by the trader. Hence another database for storing the trading decisions as well.
 - A GUI interface for the trader to view all this information on the screen.
 - ***We have a separate part to explain market data later***
-



Traditional Algo Trading System

- The traditional architecture of the entire automated trading system can now be broken down into:
 - The exchange(s) – the external world
 - The server
 - ❑ Market Data receiver
 - ❑ Store market data
 - ❑ Store orders generated by the user
 - Application
 - ❑ Take inputs from the user including the trading decisions
 - ❑ Interface for viewing the information including the data and orders
 - ❑ An order manager sending orders to the exchange

Traditional Algo Trading System





Limitations of the Traditional Algo Trading System

- DMA: Direct market access (DMA) refers to access to the electronic facilities and order books of financial market exchanges that facilitate daily securities transactions. Without intervention by a sell-side trader
 - **Limitation1: Latency**
 - traditional architecture could not scale up to the needs and demands of Automated trading with DMA. The latency between the origin of the event to the order generation went beyond the dimension of human control and entered the realms of milliseconds and microseconds. Order management also needs to be more robust and capable of handling many more orders per second. Since the time frame is minuscule compared to human reaction time, risk management also needs to handle orders in real-time and in a completely automated way.
-



Limitations of the Traditional Algo Trading System

→ **Limitation2: Scalability**

- since the architecture now involves automated logic, 100 traders can now be replaced by a single automated trading system. This adds scale to the problem. So each of the logical units generates 1000 orders and 100 such units mean 100,000 orders every second. This means that the decision-making and order sending part needs to be much faster than the market data receiver in order to match the rate of data.

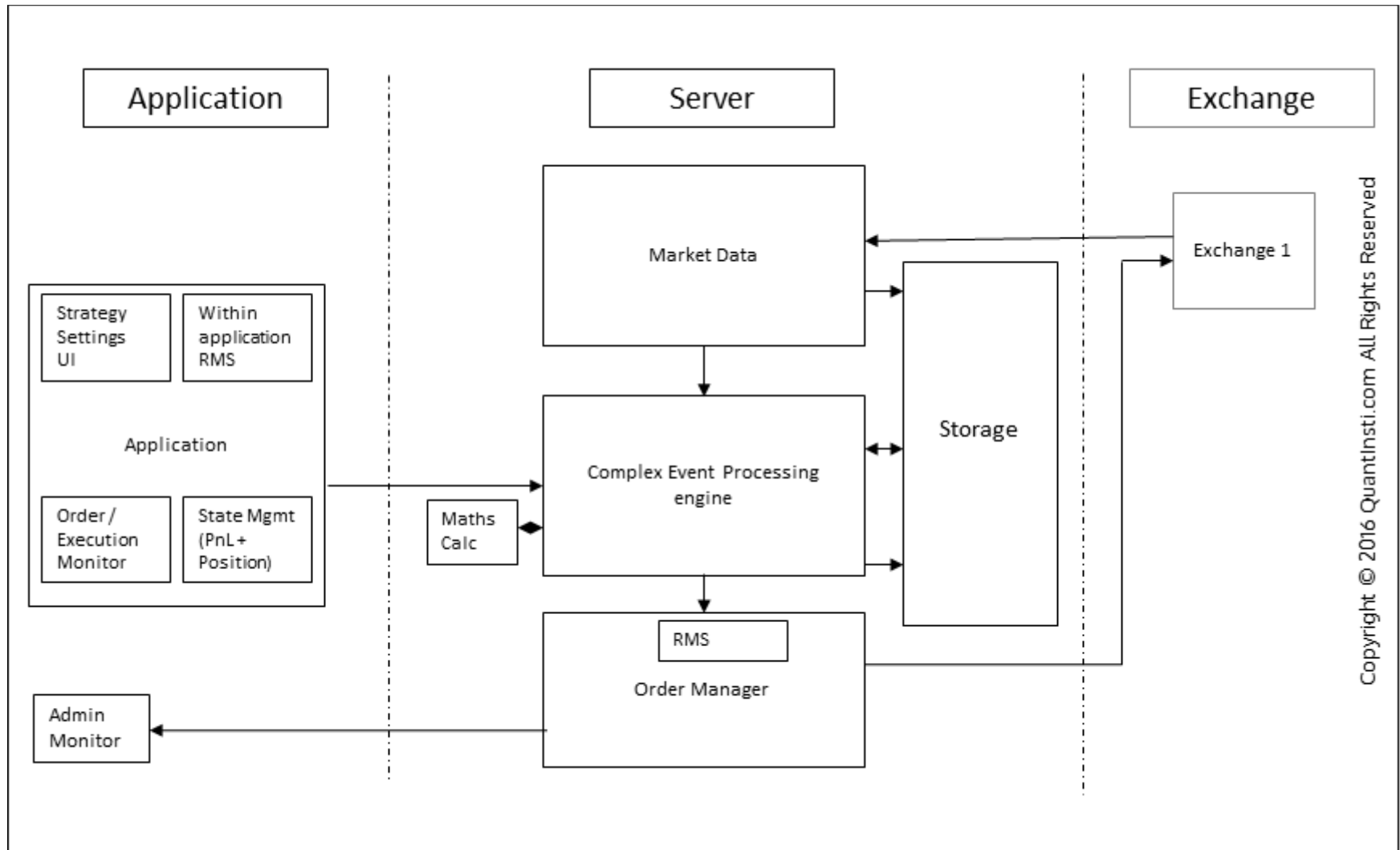


Limitations of the Traditional Algo Trading System

→ **Limitation2:** Scalability.

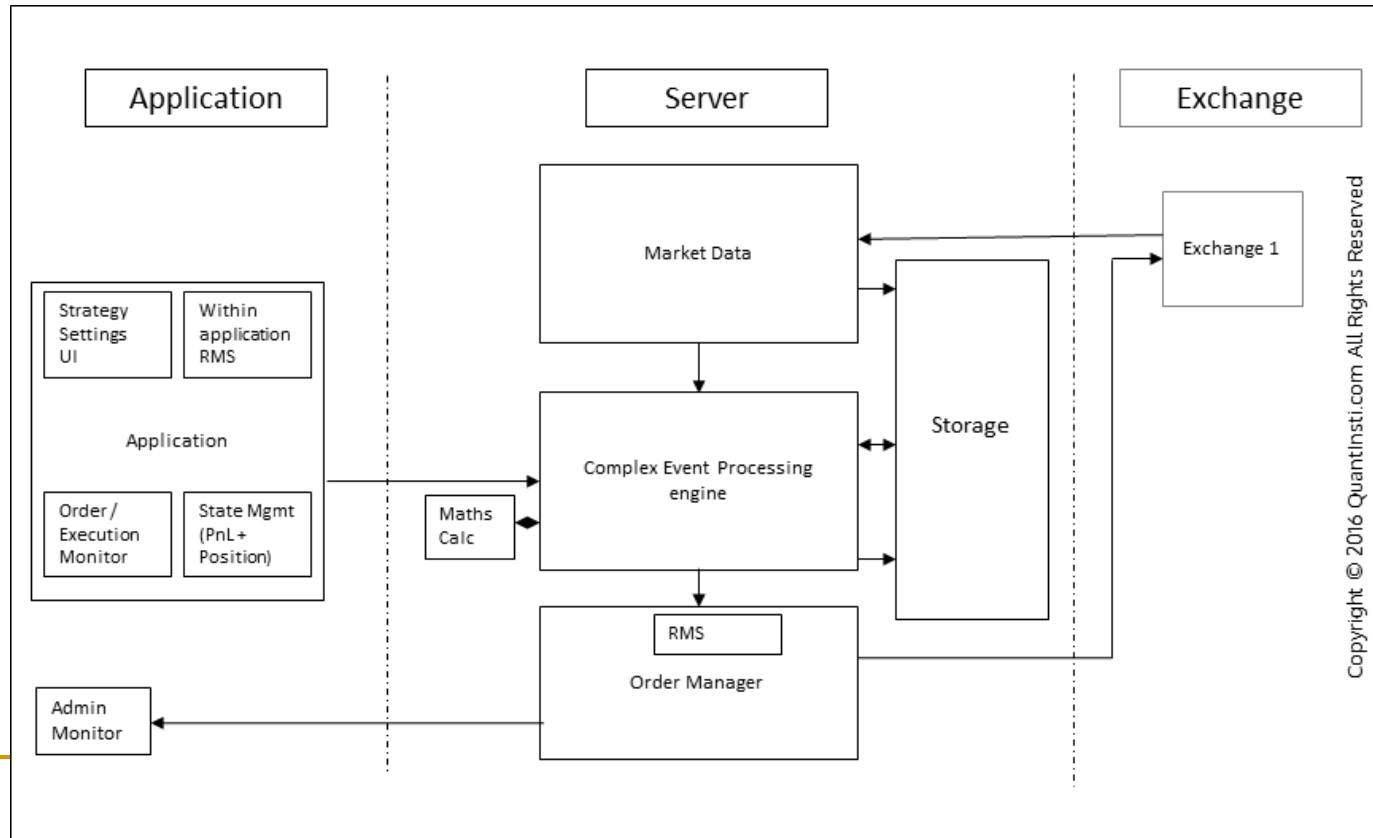
- What if your trading system should server 100 traders?
- since the architecture now involves automated logic, 100 traders can now be replaced by a single automated trading system. This adds scale to the problem. So each of the logical units generates 1000 orders and 100 such units mean 100,000 orders every second. This means that the decision-making and order sending part needs to be much faster than the market data receiver in order to match the rate of data.

New Algo Trading System Architecture



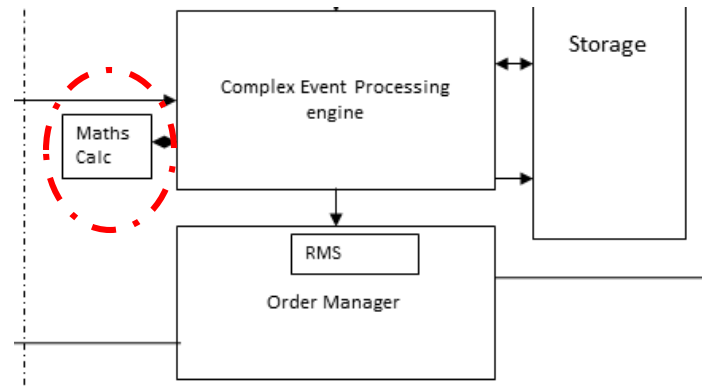
New Algo Trading System Architecture

- CEP: To overcome the limitations of the traditional system architecture, the engine which runs the **logic of decision making**, also known as the 'Complex Event Processing' engine, or CEP, moved from within the application to the server



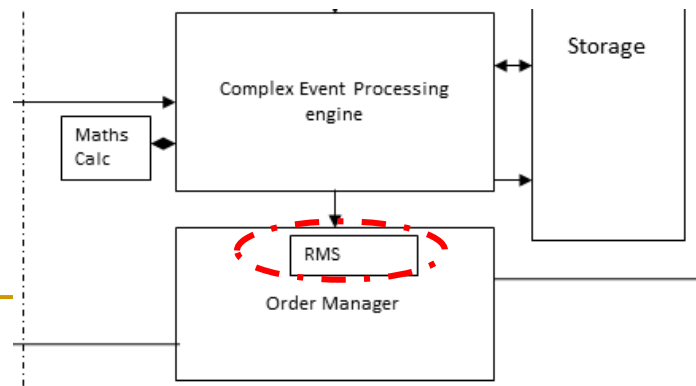
New Algo Trading System Architecture

- **Scaling:** Let us say 100 different logics are being run over a single market data event. However, there might be common pieces of complex calculations that need to be run for most of the 100 logic units, let's say, the calculation of greeks for options. If each logic were to function independently, each unit would do the same greek calculation, which would unnecessarily use up processor resources. In order to optimize on the redundancy of calculation, complex redundant calculations are typically hived off into a separate calculation engine which provides the greeks as an input to the CEP in the automated trading system.



New Algo Trading System Architecture

- **Risk Management:** The rest of the risk checks in an automated trading systems are performed now by a separate Risk Management System (RMS) within the Order Manager (OM), just before releasing an order. The problem of scale also means that where earlier there were 100 different traders managing their risk, there is now only one RMS system to manage risk across all logical units/strategies. However, some risk checks may be particular to certain strategies, and some might need to be done across all strategies. Hence the RMS itself involves strategy level RMS (SLRMS) and global RMS (GRMS). It might also involve a UI to view the SLRMS and GRMS.



New Algo Trading System Architecture

System Architecture of an Algorithmic Trading Platform





New Algo Trading System Architecture

- **Market Adapter**
 - Exchange or any market data vendor sends data in their own format. Your algorithmic trading system may or may not understand that language. Exchange provides you with an API which allows you to program and create your own adapter which can convert the format of the data into the format your system can understand.
-



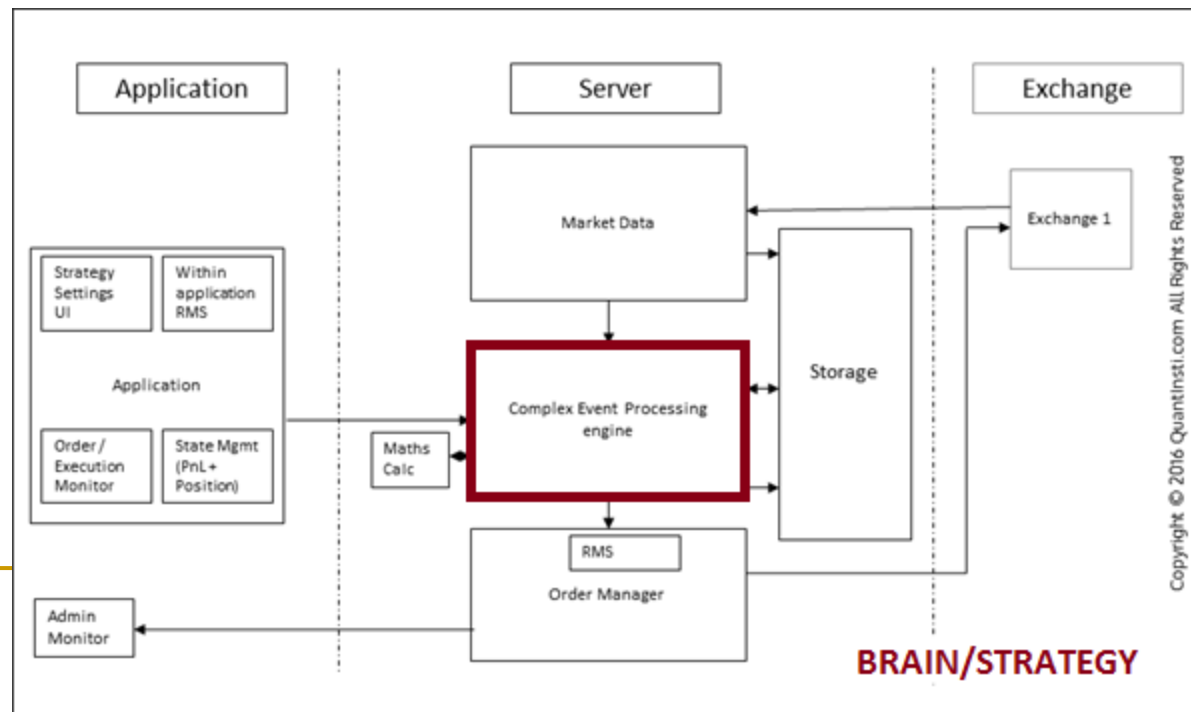
New Algo Trading System Architecture

■ **Complex Event Processing Engine**

- This part is the brain of your strategy. Once you have the data, you would need to work with it as per your strategy, which involves doing various statistical calculations, comparisons with historical data and decision making for order generation. The type of order, order quantity is prepared in this block.
 - A complex event is nothing but a set of incoming events. These include stock trends, market movements, news etc. Complex event processing is performing computational operations on complex events in a short time. In an automated trading system, the operations can include detecting complex patterns, building correlations and relationships such as causality and timing between any incoming events.
-

New Algo Trading System Architecture

- Speed: CEP systems process events in real-time, thus the faster the processing of events, the better a CEP system is. For example, if an automated trading system is designed to detect a profit-making opportunity for the next 1 second, but the time taken by the CEP system exceeds this threshold, then the trading system won't be able to make any profits.





New Algo Trading System Architecture

- The CEP system comprises of four parts:
 - ❑ CEP engine
 - ❑ CEP rules
 - ❑ CEP Web Service
 - ❑ CEP result interface
 - The two primary components of any CEP system are the CEP engine and the set of CEP rules. The CEP engine processes incoming events based on CEP rules. These rules and the events that go as an input to the CEP engine are determined by the trading system (trading strategy) applied.
 - For a quant, the majority of his work is concentrated in this CEP system block. A quant will spend most of his time in formulating trading strategies; performing rigorous backtesting, optimization, and positioning among other things. This is done to ensure the viability of the trading strategy in real markets. No single strategy can guarantee everlasting profits. Hence, quants are required to come up with new strategies on a regular basis to maintain an edge in the markets.
-



New Algo Trading System Architecture

■ **Order Routing System**

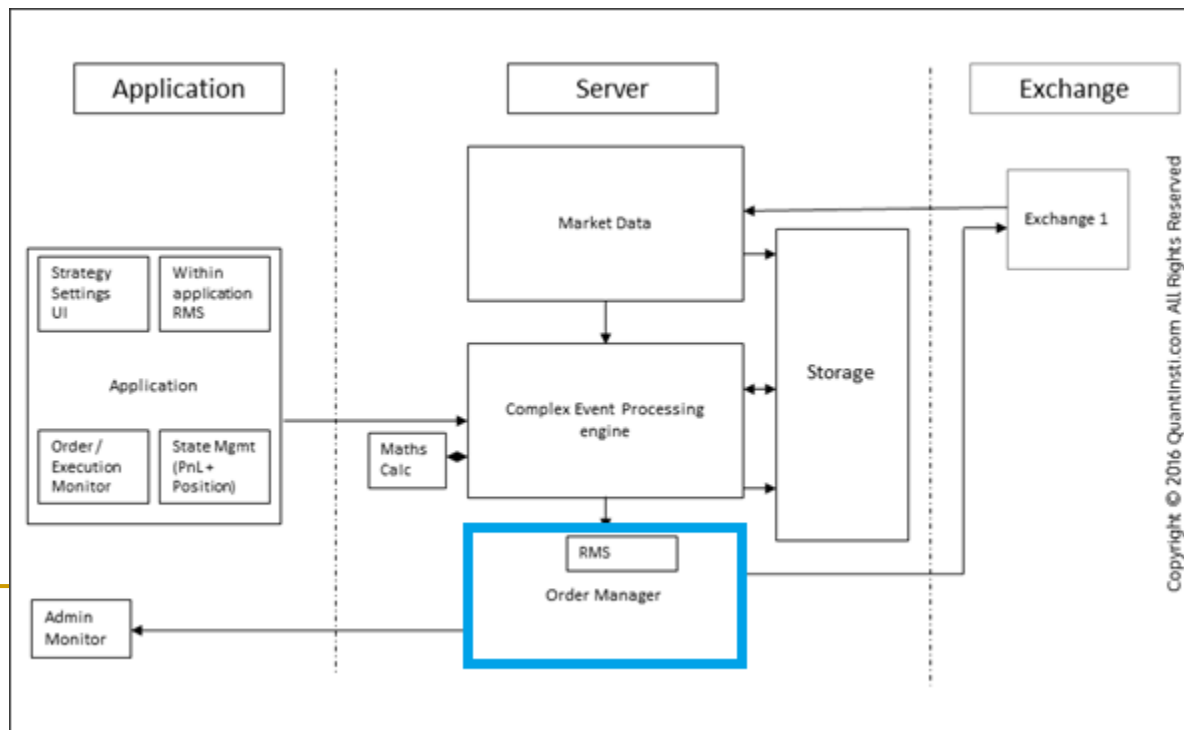
- The order is encrypted in the language which the exchange can understand, using the APIs which are provided by the exchange. There are two kinds of APIs provided by the exchange:
 - ❑ Native API. Native APIs are those which are specific to a certain exchange
 - ❑ FIX API. Native APIs are those which are specific to a certain exchange. The FIX (Financial Information Exchange) protocol is a set of rules used across different exchanges to make the data flow in security markets easier and effective..

New Algo Trading System Architecture

■ Order Routing System

- The order manager module comprises of different execution strategies which execute the buy/sell orders based on pre-defined logic. Some of the popular execution strategies include VWAP, TWAP etc. There are different processes like order routing, order encoding, transmission etc. that form part of this module.

➤ Order Management System (OMS)





New Algo Trading System Architecture

- **Risk Management in Order Manager**
- The absence of risk checks or faulty risk management can lead to enormous irrecoverable losses for a quantitative firm. Thus, a risk management system (RMS) forms a very critical component of any automated trading system.
- There are 2 places where Risk Management is handled in automated trading systems:
 - ***Within the application*** – We need to ensure those wrong parameters are not set by the trader. It should not allow a trader to set grossly incorrect values nor any fat-finger errors.
 - ***Before generating an order in OMS*** – Before the order flows out of the system we need to make sure it goes through some risk management system. This is where the most critical risk management check happens



New Algo Trading System Protocols

- **New trading system should be able to serve MULTIPLE traders to trade MULTIPLE products from MULTIPLE destinations, e.g. exchanges**
 - The new architecture was capable of scaling to many strategies per server, the need to connect to multiple destinations from a single server.
 - The order manager hosted several adaptors to send orders to multiple destinations and receive data from multiple exchanges
-



New Algo Trading System Protocols -- FIX

- Each adaptor acts as an interpreter between the protocol that is understood by the exchange and the protocol of communication within the system. Multiple exchanges would thus, require multiple adaptors.
- However, to add a new exchange to the automated trading system, a new adapter has to be designed and plugged into the architecture since each exchange follows its protocol that is optimized for features that the exchange provides. To avoid this hassle of adapter addition, standard protocols have been designed. The most prominent amongst them is the FIX protocol. This not only makes it manageable to connect to different destinations on the fly but also drastically reduces the go-to-market time when it comes to connecting with a new destination.



New Algo Trading System Protocols -- FIX

- ➔ FIX (Financial Information eXchange) protocol is the defacto standard for message communication for almost two decades of electronic trading. In 1992, Salamon brothers and Fidelity Investments initiated FIX protocol to be used in equity trading. Today, it is used by a variety of market participants, firms, and vendors. With the advent of electronic trading, various exchanges and firms devised their own messaging formats, and so FIX was seen and developed as an **intermediary messaging** format, a common underlying means of **standardized** communications.
-



FIX Benefits

- **Connectivity costs**
- Widespread use of a standard messaging protocol reduces cost and complexity of integrating various internal processes, hence beneficial to all stakeholders. There is a vast improvement in the ease of maintaining application working over FIX than one that uses a proprietary protocol. The use of FIX reduces the costs of establishing a trading platform, automated execution tool, etc. simply by allowing replication from existing implementations.



FIX Benefits

- **Reduced complexity for involving multiple partners**
- If three people are having a conversation, it makes sense for them to use the language common to all three, the same analogy can be applied to message interaction between different market participants and stakeholders. With FIX as a standard, it is easier to involve multiple partners as one avoids the added complexity of using proprietary protocols to deal with multiple firms/exchanges.
- **Reliable and Reduced Risk, and Continuous Development**
- **Increased Quality of Service**
- Investment management firms will be able to switch more easily between brokers due to the availability of a common standard protocol. Thereby, increasing competition among brokers for trading services; resulting in increased quality of service

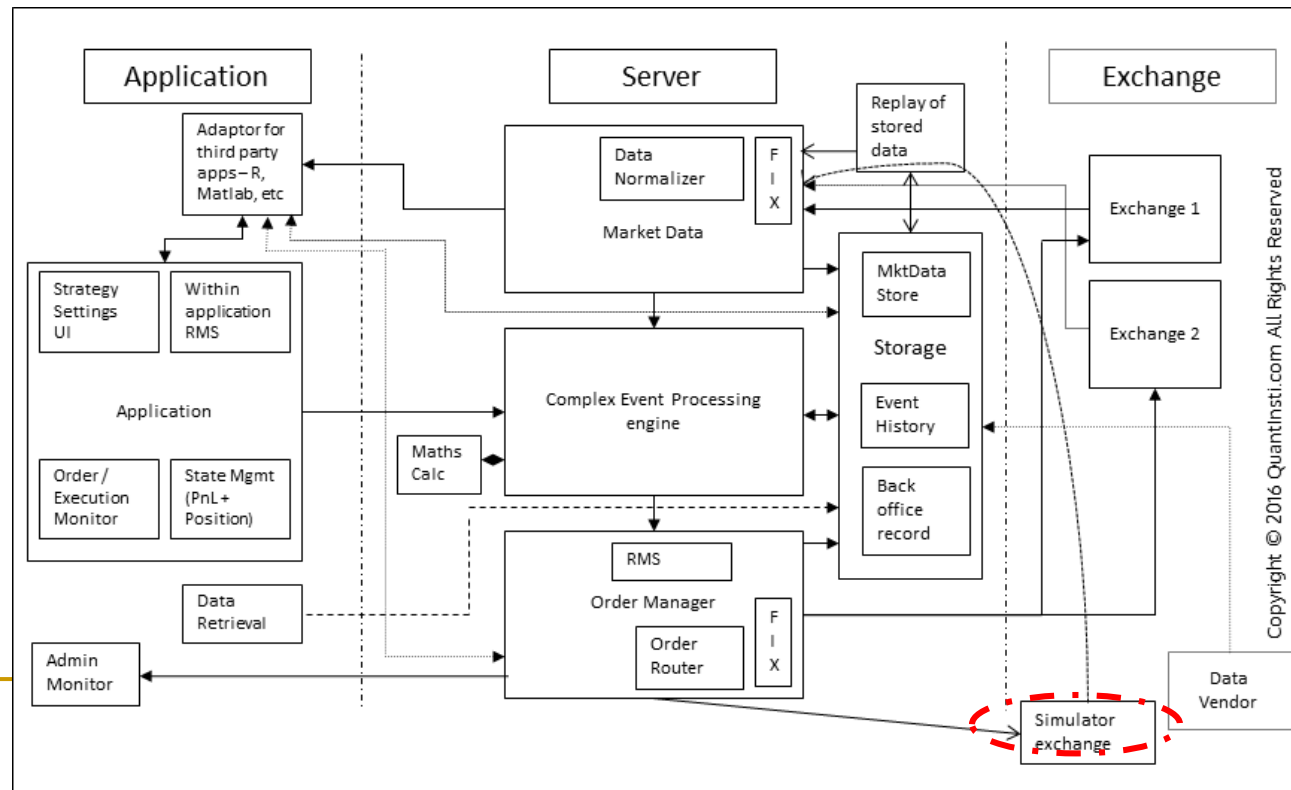
FIX Message Structure

- FIX is layered over a TCP connection. A FIX message has tag value pairs; each pair being separated with a ASCII character code SOH (0x001), a tag separated from a value with an equality character (=). Every message has a header, a body and a tail. The tail is the checksum with the tag 10. The fix version and body length of the message are in the header part (8 and 9 tags respectively). The message type is denoted by the value of tag 35.

```
8=FIX.4.19=11235=049=BRKR56=INVMGR34=23552=19980604-07:58:28112=19980604-07:58:2810=157
```

New Algo Trading System Protocols

- With FIX, simulation becomes very easy as receiving data from the real market and sending orders to a simulator is just a matter of using the FIX protocol to connect to a simulator. The simulator itself can be built in-house or procured from a third-party vendor. Similarly recorded data can be replayed with the adaptors being agnostic to whether the data is being received from the live market or from a recorded data set.





New Algo Trading System – Low Latency

- Over time, reducing latency has become a necessity for many reasons like:
 - ❑ The strategy makes sense only in a low latency environment
 - ❑ Survival of the fittest – competitors pick you off if you are not fast enough
 - ➔ The problem, however, is that latency is really an overarching term that encompasses several different delays. Although it is very easily understood, it is quite difficult to quantify. It, therefore, becomes increasingly important how the problem of reducing latency is approached.
-



New Algo Trading System – Low Latency

- If we look at the basic life cycle in an automated trading system,
 - ❑ A market data packet is published by the exchange
 - ❑ The packet travels over the wire
 - ❑ The packet arrives at a router on the server side.
 - ❑ The router forwards the packet over the network on the server side.
 - ❑ The packet arrives on the Ethernet port of the server.
 - ❑ Depending on whether this is UDP/TCP, processing takes place and the packet, stripped of its headers and trailers makes its way to the memory of the adaptor.
 - ❑ The adaptor then parses the packet and converts it into a format internal to the algorithmic trading platform
 - ❑ This packet now travels through the several modules of the system – CEP, tick store, etc.
 - ❑ The CEP analyses and sends an order request
 - ❑ The order request again goes through the reverse of the cycle as the market data packet.

New Algo Trading System – Low Latency

■ Colocations:

- In an automated trading system, high latency at any of these steps ensures a high latency for the entire cycle. Hence latency optimization usually starts with the first step in this cycle that is in our control i.e, “the packet travels over the wire”. The easiest thing to do here would be to shorten the distance to the destination by as much as possible.
- Colocations are facilities provided by exchanges to host the trading server in close proximity to the exchange. The following diagram illustrates the gains that can be made by cutting the distance.

RTT:
ROUND
TRIP TIME

Predicted round trip
latency cost of routing
over a long line for an
existing deployment

e.g. for 1000bytes at 1Gbps
(including access network)

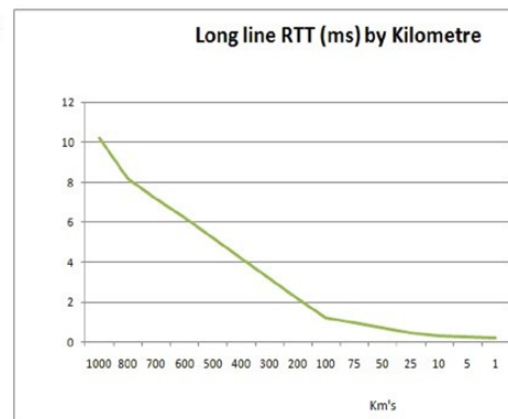
1000km = 10.1mS

100km = 1.2mS

10km = 303μS

1km = 213μS

0km = 203μS



Copyright © 2016 Quantinsti.com All Rights Reserved



New Algo Trading System – Low Latency

- **Colocations:**
- for any kind of a high-frequency strategy involving a single destination, Colocation has become a defacto must
- However, strategies that involve multiple destinations need some careful planning. Factors like the time taken by the destination to reply to order requests and its comparison with the ping time between the two destinations must be considered before making such a decision. The decision may be dependent on the nature of the strategy as well.



New Algo Trading System – Low Latency

- **Network Latency: first step**
- **Propagation Latency**
 - In an automated trading system, propagation latency signifies the time taken to send the bits along the wire, constrained by the speed of light.
 - Several optimizations have been introduced to reduce the propagation latency apart from reducing the physical distance.



New Algo Trading System – Low Latency

■ **Propagation Latency**

- For example, the estimated roundtrip time for an ordinary cable between Chicago and New York is 13.1 milliseconds. Spread Networks, in October 2012, announced latency improvements which brought the estimated roundtrip time to 12.98 milliseconds. Microwave communication was adopted further by firms such as Tradeworx bringing the estimated roundtrip time to 8.5 milliseconds. Note that the theoretical minimum is about 7.5 milliseconds. Continuing innovations are pushing the boundaries of science and fast reaching the theoretical limit of the speed of light. Latest developments in laser communication, earlier adopted in defence technologies, has further shaved off an already thinning latency by nanoseconds over short distances.



New Algo Trading System – Low Latency

- **Network processing latency**
- Network processing latency signifies the latency introduced by routers, switches, etc.
- The next level of optimization in the architecture of an automated trading system would be in the number of hops that a packet would take to travel from point A to point B. A hop is defined as one portion of the path between source and destination during which a packet doesn't pass through a physical device like a router or a switch. For example, a packet could travel the same distance via two different paths. But It may have two hops on the first path versus 3 hops on the second. Assuming the propagation delay is the same, the routers and switches each introduce their own latency and usually as a thumb rule, more the hops more is the latency added.

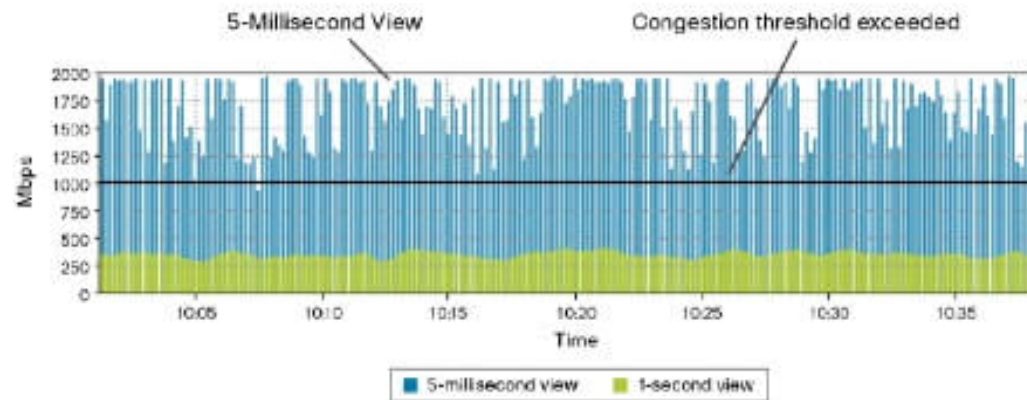
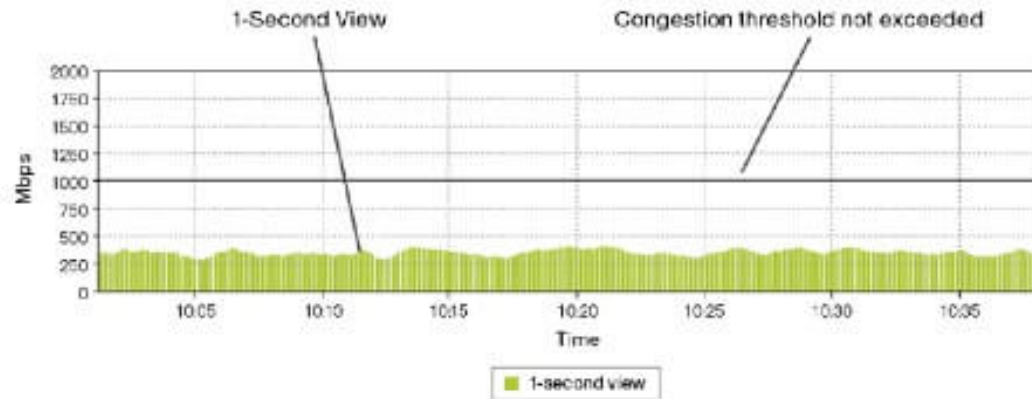


New Algo Trading System – Low Latency

- **Network processing latency**
- Network processing latency may also be affected by what we refer to as microbursts. Microbursts are defined as a sudden increase in the rate of data transfer which may not necessarily affect the average rate of data transfer. Since automated trading systems are rule-based, all such systems will react to the same event in the same way.
- As a result, a lot of participating systems may send orders leading to a sudden flurry of data transfer between the participants and the destination leading to a microburst.
- The following diagram represents what a microburst is.

New Algo Trading System – Low Latency

■ Network processing latency





New Algo Trading System – Low Latency

- **Network processing latency**

- The first figure shows a 1-second view of the data transfer rate. We can see that the average rate is well below the bandwidth available of 1Gbps. However, if we dive deeper and look at the second image (the 5-millisecond view), we see that the transfer rate has spiked above the available bandwidth several times each second. As a result, the packet buffers on the network stack, both in the network endpoints and routers and switches may overflow. To avoid this, typically a bandwidth that is much higher than the observed average rate is usually allocated for an automated trading system.



New Algo Trading System – Low Latency

- **Serialization latency**
- Serialization latency for an automated trading system signifies the time taken to pull the bits on and off the wire.
- A packet size of 1500 bytes transmitted on a T1 line (1,544,000 bps) would produce a serialization delay of about 8 milliseconds. However, the same 1500 byte packet using a 56K modem (57344bps) would take 200 milliseconds. A 1G Ethernet line would reduce this latency to about 11 microseconds.



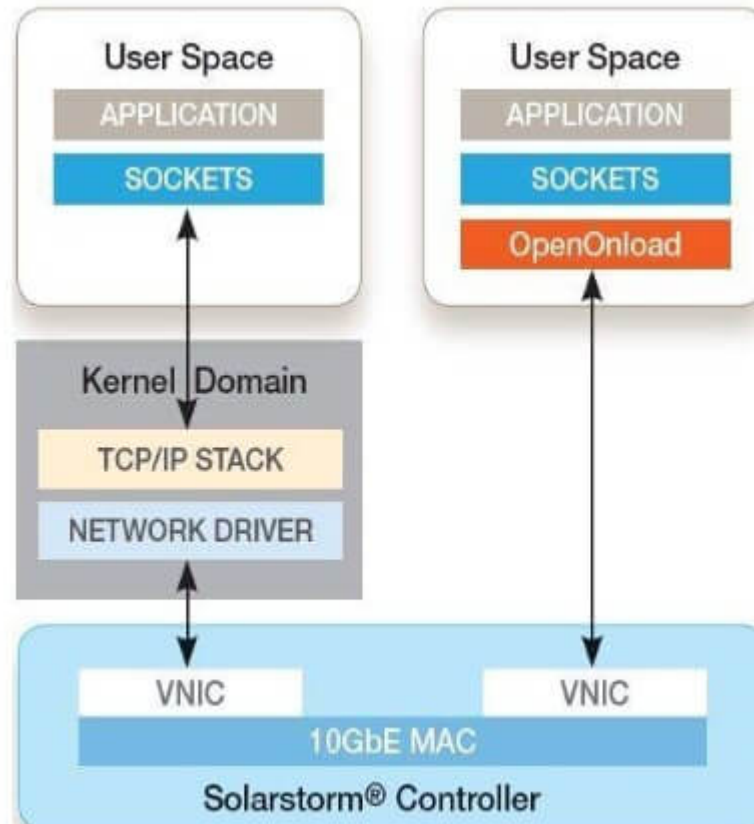
New Algo Trading System – Low Latency

■ **Interrupt latency**

- Interrupt latency in an automated trading system signifies a latency introduced by interrupts while receiving the packets on a server.
- Interrupt latency is defined as the time elapsed between when an interrupt is generated to when the source of the interrupt is serviced. When is an interrupt generated? Interrupts are signals to the processor emitted by hardware or software indicating that an event needs immediate attention. The processor, in turn, responds by suspending its current activity, saving its state and handling the interrupt. Whenever a packet is received on the NIC, an interrupt is sent to handle the bits that have been loaded into the receive buffer of the NIC. The time taken to respond to this interrupt not only affects the processing of the newly arriving payload, but also the latency of the existing processes on the processor.

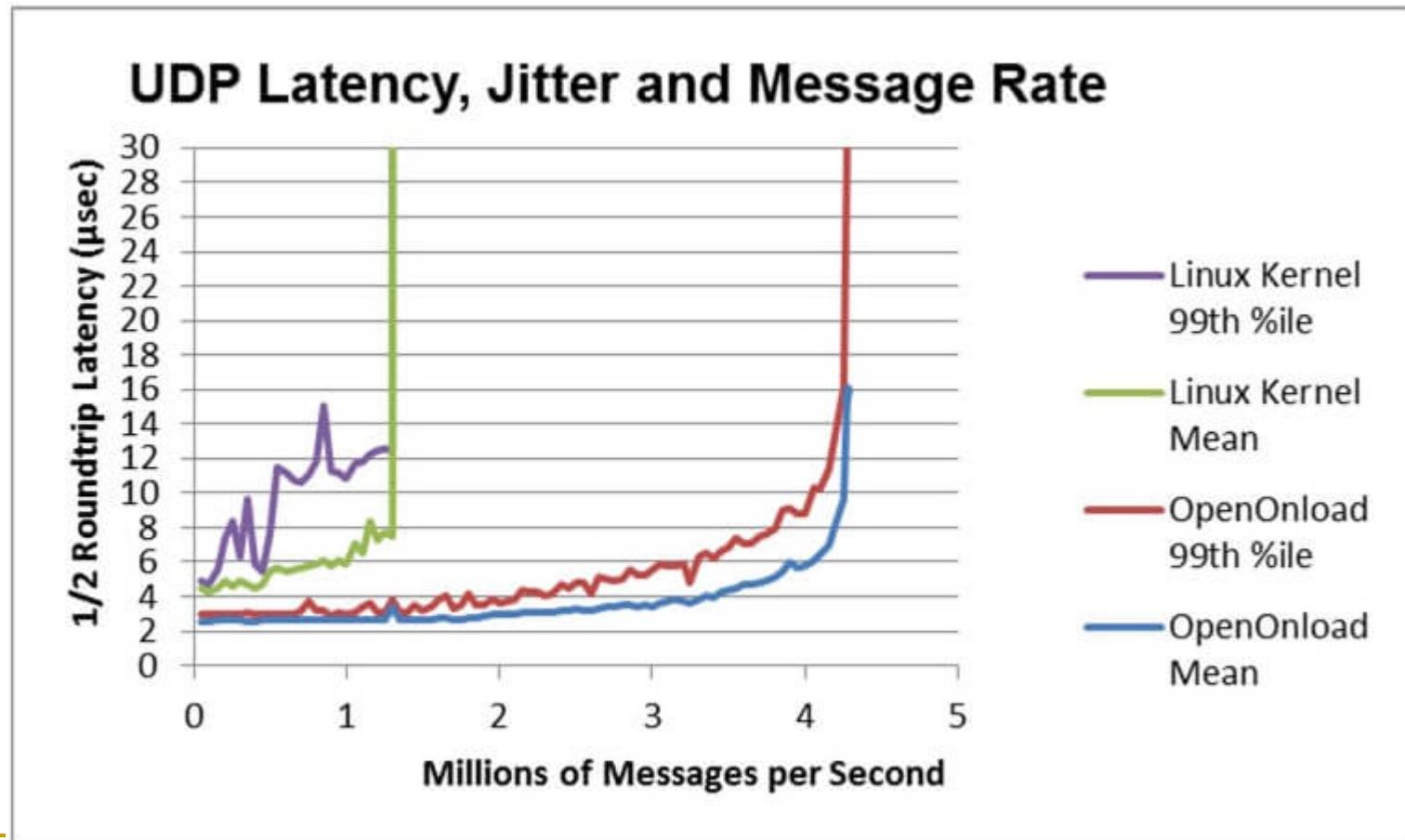
New Algo Trading System – Low Latency

- **Interrupt latency**
- Solarflare introduced “OpenOnload” in 2011, which implements a technique known as kernel bypass, where the processing of the packet is not left to the operating system kernel but to the userspace itself. The entire packet is directly mapped into the userspace by the NIC and is processed there. As a result, interrupts are completely avoided.



New Algo Trading System – Low Latency

- Thus, the rate of processing each packet is accelerated. The following diagram clearly demonstrates the advantages of kernel bypass.





New Algo Trading System – Low Latency

- **Application latency**

- Application latency for an automated trading system signifies the time taken by the application to process.
- This is dependent on the several packets, the processing allocated to the application logic, the complexity of the calculation involved, programming efficiency etc. Increasing the number of processors on the system would, in general, reduce the application latency. Same is the case with increased clock frequency. A lot of automated trading systems take advantage of dedicating processor cores to essential elements of the application like the strategy logic for eg. This avoids the latency introduced by the process switching between cores.

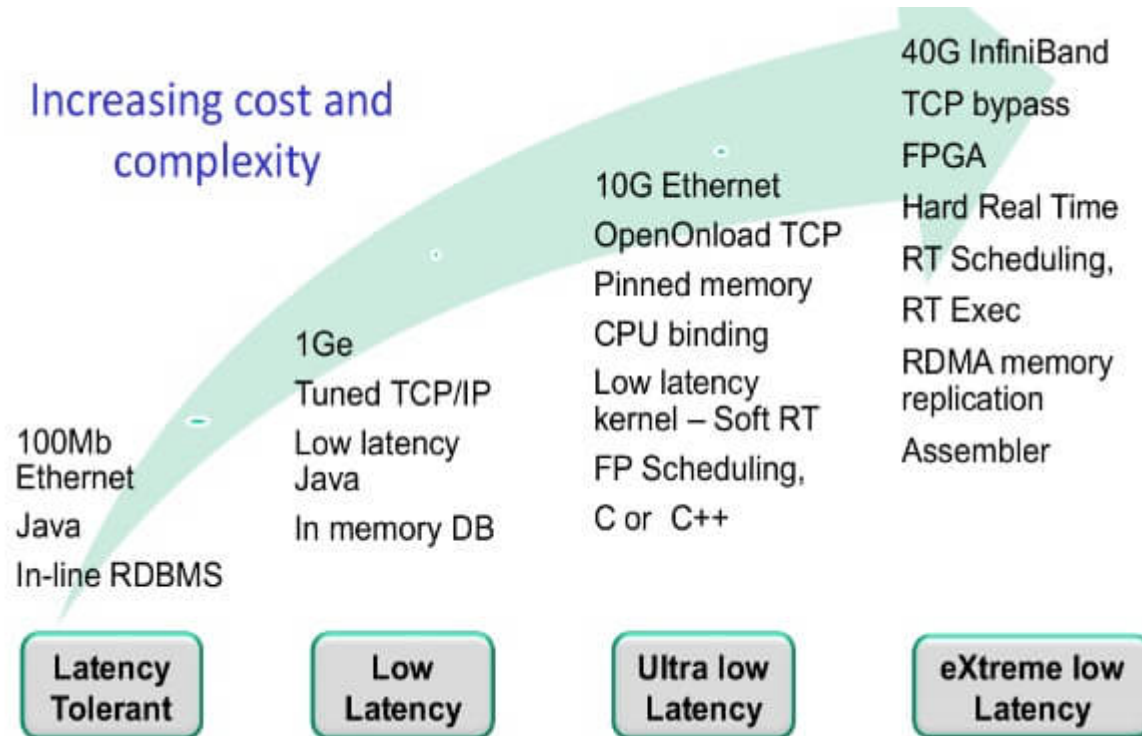


New Algo Trading System – Low Latency

- **Application latency**
- Similarly, if the programming of the strategy in an automated trading system has been done keeping in mind the cache sizes and locality of memory access, then there would be a lot of memory cache hits resulting in further reduction of latency. To facilitate this, a lot of systems use very low-level programming languages to optimize the code to the specific architecture of the processors. Some firms have even gone to the extent of burning complex calculations onto hardware using Fully Programmable Gate Arrays (FPGA). With increasing complexity comes increasing cost and the following diagram aptly illustrates this.

New Algo Trading System – Low Latency

■ Application latency





New Algo Trading System – Low Latency

- **Levels of sophistication**
- The world of high-frequency algorithmic trading has entered an era of intense competition. With each participant adopting new methods of ousting the competition, technology has progressed by leaps and bounds. Modern-day algorithmic trading architectures are quite complex compared to their early-stage counterparts. Accordingly, advanced automated trading systems are more expensive to build both in terms of time and money.



New Algo Trading System – Low Latency

■ Levels of sophistication



	Standard 10GE network card	Low latency 10GE network card	FPGA	ASIC
Latency	20 microseconds + application time	5 microseconds + application time	3-5 microseconds	Sub microsecond latency
Ease of deployment	Trivial	Kernel driver installation	Retraining of programmers	Specialists
Man years effort to develop	Weeks	Months	2-3 man-years	2-3 man-years



Git – IMPORTANT!

- **Version Control Tool / Source Code Management System**
- **Everyone should register a github account <https://github.com/> or bitbucket**
 - <https://guides.github.com/activities/hello-world/>
- **Repository**
 - *DON'T PUT LARGE DATASET THERE. Typically not large*
- ***Snapshot: store it as BLOB(binary large object)***
- **Local**
 - Everything on your current PC
- **Remote**
 - Cloud Server, e.g. github,



- **Linux command**

- **Why Linux?**

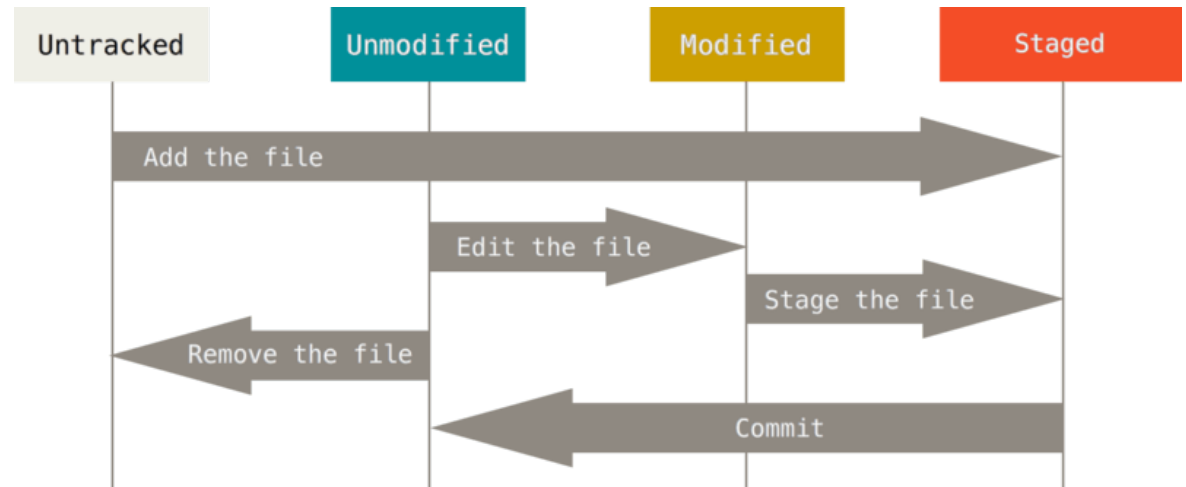
- No GUI issue. Clean. Not much dependency
 - Stable. not easy to crash
 - efficient

- **Self study please!**

- **SSH Protocol**

Git – Keep track of the changes

- <https://git-scm.com/book/en/v2> : chapter 1, 2
- git clone
- git init
- git add
- git status
- git commit –m ‘update’
- git log
- git checkout
- Remote related
 - ❑ git pull
 - git fetch: similar with git pull, but just fetch the remote data. No automatically merge
 - ❑ git remote
 - ❑ git push



Git – Manage Branches

- <https://git-scm.com/book/en/v2> : chapter 3

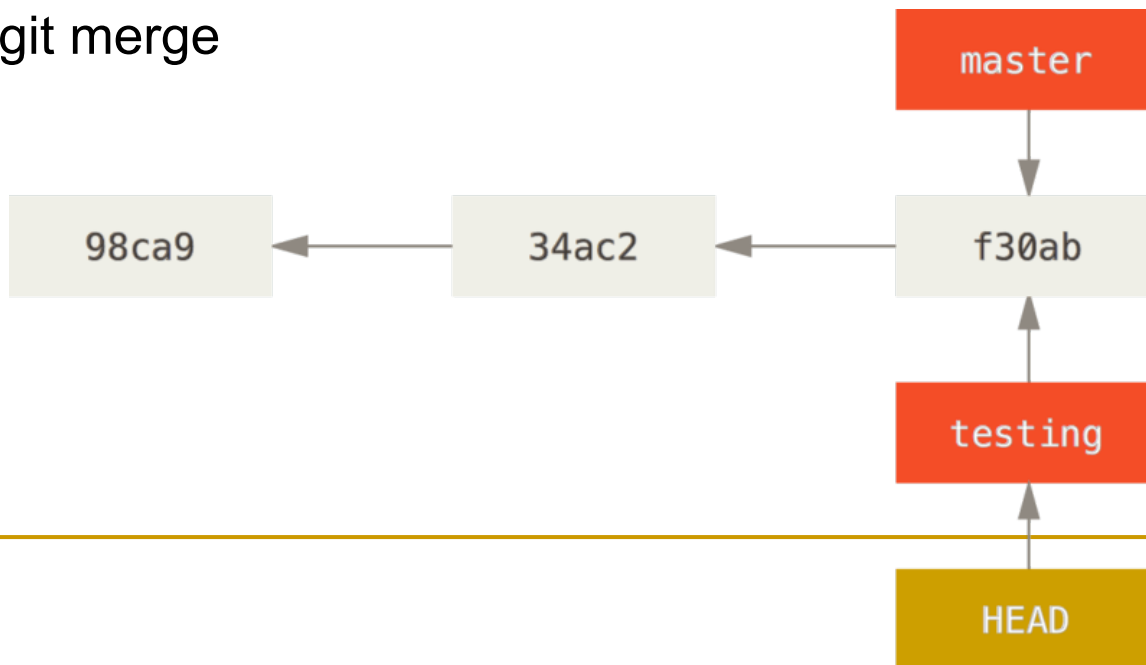
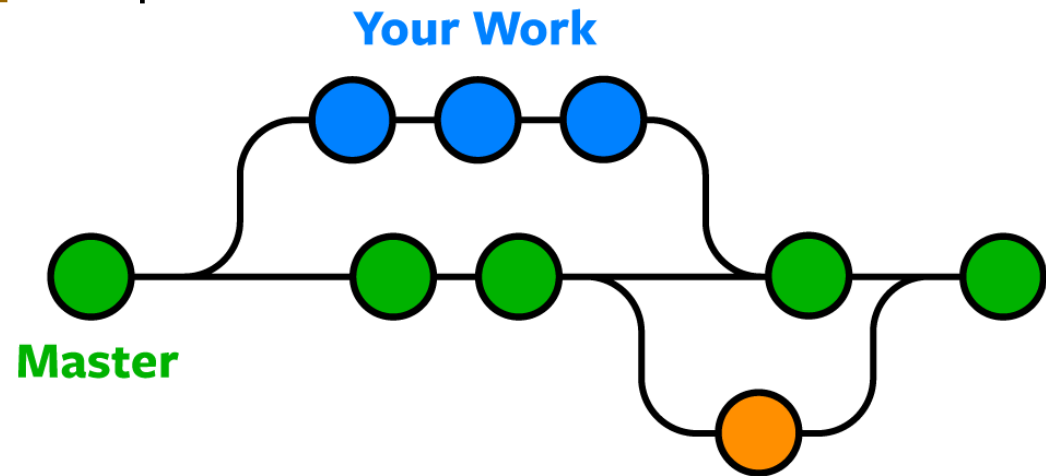
- git branch

- git checkout

- ❑ go back to previous version
- ❑ apply HEAD pointer to point to the new branch **testing** as shown In the picture below

- git diff

- git merge





Git Collaboration

- Documentation
 - <https://www.atlassian.com/git/tutorials/syncing>
 - Github flow: <https://guides.github.com/introduction/flow/>
 - Pull Request (PR)
 - Steps:
 - ❑ A developer creates the feature in a dedicated branch in their local repo.
 - ❑ The developer pushes the branch to a public Github/Bitbucket repository.
 - ❑ The developer files a pull request via Github/Bitbucket.
 - ❑ The rest of the team reviews the code, discusses it, and alters it.
 - ❑ The project maintainer merges the feature into the official repository and closes the pull request.
-

Thanks!

Reference

- <https://www.investopedia.com/articles/active-trading/101014/basics-algorithmic-trading-concepts-and-examples.asp>
- <https://www.investopedia.com/terms/a/algorithmictrading.asp>
- <https://blog.quantinsti.com/automated-trading-system/>
- <http://www.ruanyifeng.com/blog/2015/12/git-workflow.html>